

SIGCB-QR

Sistema Integral de Gestión de Biblioteca con Códigos QR

Informe técnico — Entrega Final

Arias Moreira, Maybelin Gregoria^{*}

Romero Méndez, Bryam Steven^{**}

Zambrano Moreira, Alison Ariana^{***}

Universidad Técnica Estatal de Quevedo (UTEQ)

3 de septiembre de 2026

Versión 1.0.0

github.com/a112i3s4o5n6/-SIGCB-QR-Biblioteca

Commit `8503a5f` Digest SHA-256 `3555a8d8a2defc10...`

DOI: *pendiente de depósito en Zenodo*

^{*} ORCID: pendiente de registro.

^{**} ORCID: pendiente de registro.

^{***} ORCID: pendiente de registro.

Resumen

Se presenta SIGCB-QR, un sistema de gestión bibliotecaria universitaria automatizada mediante códigos QR, construido como API REST en Spring Boot 3.5 con un frontend Laravel 13 que actúa como *Backend for Frontend*, sobre PostgreSQL 16 y Redis 7. El sistema cubre autenticación JWT en cookie `HttpOnly`, catálogo, inventario, préstamos, devoluciones, reservas, multas y auditoría, con control de acceso basado en roles y trece procedimientos almacenados. La investigación adopta la metodología *Design Science Research* de Peffers y se enmarca en cuatro preguntas de investigación. El informe documenta la arquitectura y sus decisiones (nueve ADR), y sobre todo el **estudio empírico** realizado sobre el sistema en marcha: una auditoría de seguridad de 42 comprobaciones basada en OWASP API Security Top 10, la medición de cobertura con JaCoCo, cuatro corridas de carga con k6, una auditoría de frontend con Lighthouse y un instrumento SUS de usabilidad.

El resultado principal no son las cifras favorables, sino los **tres defectos** que las auditorías descubrieron en un sistema cuya suite de pruebas se daba por correcta: el endpoint del catálogo devolvía 500 en todo acierto de caché, toda ruta inexistente devolvía 500 filtrando mensajes internos del framework, y la política de seguridad de contenido bloqueaba los iconos, la tipografía y el módulo de códigos QR de la propia aplicación. Ninguno era detectable midiendo cobertura. Se realiza un cribado de trabajos relacionados siguiendo el flujo PRISMA sobre el índice de Crossref, del que resultan ocho estudios incluidos, y se delimita la brecha: ninguno de ellos separa la API del dominio mediante un *Backend for Frontend*, ninguno documenta control de acceso por roles sobre los endpoints de escritura y ninguno publica datos crudos que permitan reproducir sus cifras. Se declaran las amenazas a la validez de cada medición conforme a Wohlin, Cook y Campbell, y se documenta un caso en que el propio instrumento de medida produjo un resultado falso: el arnés de carga medía el limitador de tasa en lugar del catálogo. Se declara asimismo lo que este trabajo no consiguió medir, señaladamente la usabilidad, cuyo instrumento quedó construido y validado pero se aplicó a cero participantes.

Palabras clave: gestión bibliotecaria, códigos QR, arquitectura de software, ADR, OWASP, evidencia empírica, reproducibilidad, validez.

Abstract. SIGCB-QR is a university library management system whose bibliographic loan cycle is automated through QR codes, built as a REST API in Spring Boot 3.5 with a Laravel 13 frontend acting as a Backend for Frontend on top of PostgreSQL 16 and Redis 7. The system covers HttpOnly-cookie JWT authentication, catalog, inventory, loans, returns, reservations, fines and auditing, with role-based access control and thirteen stored procedures. The research follows the Design Science Research methodology of Peffers and is guided by four research questions. Beyond documenting the architecture and its nine architecture decision records (ADRs), the report focuses on the empirical study run on the live system: a 42-check security audit based on OWASP API Security Top 10, JaCoCo test coverage measurement, four k6 load runs, a Lighthouse frontend audit and a System Usability Scale (SUS) instrument. The main finding is not the favourable metrics but the three real defects the audits uncovered in a system whose test suite was assumed to be correct: the catalog endpoint returned 500 on every cache hit, every unknown route returned 500 leaking internal framework messages, and the Content Security Policy blocked the application's own icons, fonts and QR module. A PRISMA-based systematic review of related work, the identified research gap, and validity threats according to Wohlin, Cook and Campbell are discussed. The study sources are all synthetic; reported figures are reproducible, and unmeasured cases are explicitly declared as such.

Keywords: library management, QR codes, software architecture, ADR, OWASP, empirical evidence, reproducibility, validity.

Índice

Glosario de siglas

1. Introducción	6
1.1. Contexto y problema	6
1.2. Objetivos	6
1.3. Estructura del informe	6
2. Marco Teórico	6
2.1. Arquitectura de aplicaciones web	6
2.2. Servicios REST y autenticación JWT	6
2.3. Códigos QR en bibliotecas	7
2.4. Caché distribuida con Redis	7
3. Trabajos Relacionados	7
3.1. Estrategia de búsqueda (PRISMA)	7
3.2. Tabla comparativa y brecha	8
4. Metodología	9
4.1. Preguntas de investigación	9
4.2. Diseño de investigación	9
4.3. Muestreo y análisis	9
4.4. Reglas de reporte	10
4.5. Entorno	10
5. Ingeniería de Requisitos	10
5.1. Proceso y marco normativo	10
5.2. Catálogo de requisitos	10
5.3. Corrección de una deficiencia de la entrega anterior	11
5.4. Trazabilidad ejecutable	11
5.5. Calidad del producto según ISO/IEC 25010	11
6. Arquitectura	12
6.1. Visión general	12
6.2. Por qué dos pilas: el patrón Backend for Frontend	12
6.3. Decisiones registradas	15
6.4. Seguridad	16
6.5. Acceso a datos	16
7. Bloque 1: auditoría de seguridad	16
7.1. Instrumento	16
7.2. Resultados	16
7.3. Limitaciones declaradas	17
8. Bloque 2: pruebas y cobertura	17
8.1. Estado de partida	17
8.2. Resultados tras la corrección	17
8.3. Interpretación	17
9. Bloque 3: rendimiento	18
9.1. Un defecto del instrumento, corregido antes de medir	18
9.2. Resultados	18
9.3. Un resultado contrario a la hipótesis	18
10. Bloque 4: calidad del frontend	18
10.1. Por qué el rendimiento <i>bajó</i> al corregir un defecto	19

11. Defectos encontrados	19
11.1. D1 — El acierto de caché devolvía 500	19
11.2. D2 — Toda ruta inexistente devolvía 500	19
11.3. D3 — La CSP bloqueaba los recursos de la propia aplicación	20
11.4. Lectura conjunta	20
12. Operación y despliegue	20
12.1. Estado real: qué está desplegado	20
12.2. Procedimiento de despliegue	20
12.3. Operación	21
12.4. Respaldo y restauración	21
13. Discusión: respuesta a las preguntas de investigación	21
13.1. RQ1 — Reducción del tiempo de procesamiento del préstamo	21
13.2. RQ2 — Nivel de seguridad de JWT en cookie <code>HttpOnly</code> con RBAC	21
13.3. RQ3 — Efecto de la caché Redis en el rendimiento del catálogo	22
13.4. RQ4 — Usabilidad percibida	22
13.5. Lectura de conjunto	22
14. Amenazas a la validez	23
14.1. Validez de constructo	23
14.2. Validez interna	23
14.3. Validez externa	23
14.4. Validez de conclusión	24
15. Ética y tratamiento de datos	24
16. Conclusiones	24
16.1. Trabajo futuro	25
17. Declaraciones	25
1. Contribuciones de los autores (CRediT)	25
2. Conflicto de intereses	25
3. Financiación	25
4. Disponibilidad de datos y código	26
5. Consentimiento informado y ética	26
6. Uso de inteligencia artificial	26
7. Limitaciones declaradas	26
8. Reproducibilidad	26
9. Licencia	26
A. Anexos	27
A.1. Anexo A — Bitácora de observaciones	27
A.2. Anexo B — Índice de artefactos verificables	27
A.3. Anexo C — Registros de decisiones de arquitectura	27
A.4. Anexo D — Listas de comprobación	27

Índice de figuras

1.	C4 nivel 1 — Contexto. Los tres roles del sistema y los dos almacenes externos. Redis figura como caché y no como intermediario de mensajes, que es el uso que efectivamente se le da (ADR-0006).	13
2.	C4 nivel 2 — Contenedores. La frontera entre el BFF Laravel y la API Spring Boot es la que justifica el ADR-0002 : el frontend no tiene acceso a PostgreSQL.	14
3.	C4 nivel 3 — Componentes de la API. Se distingue el acceso elemental por repositorios JPA del acceso por procedimientos almacenados, que es la distinción que fija el ADR-0005	15

Índice de cuadros

1.	Siglas empleadas en este informe.	5
2.	Cribado PRISMA. Los ocho DOI incluidos se validaron uno a uno contra <code>api.crossref.org</code> y los ocho resuelven.	7
3.	Comparación de los ocho estudios incluidos y la brecha que SIGCB-QR aborda. Los ocho tienen DOI resoluble; ninguna fila es una atribución aproximada ni un ejemplo ilustrativo.	8
4.	Correspondencia Goal–Question–Metric y estado real de cada métrica. Dos de las cuatro preguntas quedan sin responder, y se declara así.	9
5.	Entorno de medición. Las imágenes están ancladas por digest SHA256 (ADR-0007), de modo que las corridas son comparables entre sí.	10
6.	Distribución de los requisitos especificados y su estado de verificación, según el estado declarado en la matriz de trazabilidad.	11
7.	Correspondencia entre ISO/IEC 25010, los requisitos que concretan cada característica y la evidencia disponible. Se declara explícitamente lo no medido.	12
8.	Decisiones de arquitectura y su coste. Detalle en <code>docs/adr/</code>	15
9.	Auditoría OWASP. Salida cruda en <code>docs/mediciones/seguridad/owasp-audit-raw.txt</code>	16
10.	Cobertura JaCoCo sobre 41 pruebas, 0 fallos. Umbral configurado: 30 % de líneas.	17
11.	Corridas de k6, en orden cronológico real. Umbral REQ-NF-001: $p95 \leq 200$ ms. Salidas crudas en <code>docs/mediciones/perf/</code>	18
12.	Lighthouse antes y después de corregir la política de seguridad de contenido. La cifra válida es la de la derecha.	19
13.	Asignación de roles CRediT.	25
14.	Artefactos verificables y comando que los regenera o comprueba.	27

Listings

1.	Topología de despliegue en una línea	12
----	--	----

Glosario de siglas

Sigla	Significado
ADR	<i>Architecture Decision Record</i> ; registro de decisión de arquitectura
API	<i>Application Programming Interface</i>
ASVS	<i>Application Security Verification Standard</i> (OWASP)
BFF	<i>Backend for Frontend</i>
CI	<i>Continuous Integration</i> ; integración continua
CRedit	<i>Contributor Roles Taxonomy</i>
CSP	<i>Content Security Policy</i>
CRUD	<i>Create, Read, Update, Delete</i>
DOI	<i>Digital Object Identifier</i>
DSR	<i>Design Science Research</i>
FAIR	<i>Findable, Accessible, Interoperable, Reusable</i>
GQM	<i>Goal-Question-Metric</i>
HSTS	<i>HTTP Strict Transport Security</i>
INCOSE	<i>International Council on Systems Engineering</i>
INVEST	<i>Independent, Negotiable, Valuable, Estimable, Small, Testable</i>
JPA	<i>Jakarta Persistence API</i>
JPQL	<i>Jakarta Persistence Query Language</i>
JWT	<i>JSON Web Token</i>
MADR	<i>Markdown Any Decision Records</i>
OCR	<i>Optical Character Recognition</i>
ORCID	<i>Open Researcher and Contributor ID</i>
ORM	<i>Object-Relational Mapping</i>
OWASP	<i>Open Worldwide Application Security Project</i>
PRISMA	<i>Preferred Reporting Items for Systematic Reviews and Meta-Analyses</i>
QR	<i>Quick Response</i> (código bidimensional)
RBAC	<i>Role-Based Access Control</i>
REST	<i>Representational State Transfer</i>
RFC	<i>Request for Comments</i>
RQ	<i>Research Question</i> ; pregunta de investigación
SRS	<i>Software Requirements Specification</i>
SUS	<i>System Usability Scale</i>
VU	<i>Virtual User</i> ; usuario virtual (k6)
ZAP	<i>Zed Attack Proxy</i> (OWASP)

Cuadro 1: Siglas empleadas en este informe.

1. Introducción

1.1. Contexto y problema

La gestión manual del préstamo bibliotecario en la UTEQ obliga a registrar en papel o en hojas de cálculo cada entrega y devolución de ejemplares, lo que dificulta conocer la disponibilidad real del acervo, aplicar el reglamento de multas de forma consistente y obtener indicadores de uso.

SIGCB-QR automatiza autenticación, catálogo, inventario, préstamos, devoluciones, reservas, multas, códigos QR, auditoría y reportes.

1.2. Objetivos

- O1.** Construir un sistema web funcional que cubra el ciclo completo del préstamo bibliotecario.
- O2.** Aplicar ingeniería de requisitos conforme a ISO/IEC/IEEE 29148:2018 [2], con trazabilidad verificable entre requisito, código, prueba y evidencia.
- O3.** Documentar las decisiones de arquitectura con sus alternativas y su coste, siguiendo la práctica de los ADR [27].
- O4.** Producir **evidencia empírica reproducible** de seguridad, cobertura, rendimiento y calidad del frontend.
- O5.** Garantizar la reproducibilidad del entorno de medición [10].

1.3. Estructura del informe

La sección 6 describe la arquitectura y sus decisiones. La sección 4 expone el método empírico. Las secciones 7 a 10 presentan los cuatro bloques de medición. La sección 11 analiza los tres defectos encontrados. La sección 14 declara las amenazas a la validez, y la 16, las conclusiones.

2. Marco Teórico

2.1. Arquitectura de aplicaciones web

La arquitectura cliente-servidor distribuye las tareas entre proveedores de recursos (servidores) y demandantes (clientes). En las aplicaciones web modernas este paradigma ha madurado hacia el patrón *Backend for Frontend* (BFF) [23], en que una capa intermedia orquesta las peticiones del navegador hacia la API de dominio, ocultando el modelo de datos y aplicando el principio de ocultación de información de Parnas [30].

SIGCB-QR adopta este BFF: un frontend Laravel que renderiza vistas y delega todo acceso a datos en una API REST Spring Boot. La separación de responsabilidades encierra la lógica de negocio, la seguridad y la persistencia en el servidor de aplicación, manteniendo la presentación ligera.

2.2. Servicios REST y autenticación JWT

REpresentational State Transfer (REST) es un estilo arquitectónico basado en seis restricciones, entre ellas la interfaz uniforme y el sistema sin estado. La API de SIGCB-QR expone recursos como `/api/libros` o `/api/prestamos` manipulados con los verbos HTTP (GET, POST, PUT, DELETE).

JSON Web Token (JWT) [18] permite transmitir información de forma segura mediante un token firmado. En SIGCB-QR el JWT se transporta en una cookie `HttpOnly` con `SameSite=Strict`, mitigando el robo de sesión por JavaScript y la falsificación de petición entre sitios [9].

2.3. Códigos QR en bibliotecas

Los códigos QR (*Quick Response*) son códigos de barras bidimensionales que permiten identificar ejemplares, acelerar el préstamo y vincular recursos digitales mediante escaneo móvil. En SIGCB-QR se usan para asociar un ejemplar a su identificador y automatizar la circulación, de modo que su aplicación responde a la primera pregunta de investigación (sección 4).

2.4. Caché distribuida con Redis

Redis es un almacén de estructuras en memoria usado como caché distribuida. En SIGCB-QR actúa exclusivamente como caché del catálogo mediante el patrón *cache-aside* de Spring Cache. Su beneficio se evalúa empíricamente con la tasa de aciertos y la latencia (sección 9), sin asumir a priori que mejora el rendimiento a cualquier escala.

3. Trabajos Relacionados

3.1. Estrategia de búsqueda (PRISMA)

Para situar el trabajo se realizó un cribado siguiendo el flujo PRISMA. La búsqueda se ejecutó el 3 de septiembre de 2026 sobre el índice de Crossref, a través de su API pública, que es la fuente que este equipo puede consultar de forma reproducible y sin suscripción institucional. Las cadenas empleadas fueron:

```
C1: QR code library management system
C2: QR code academic library services
C3: web based library information system QR code borrowing
C4: library book borrowing system QR code design implementation
```

El procedimiento y su recuento se resumen en la tabla 2.

Fase PRISMA	Registros
Identificación: resultados devueltos por las cuatro cadenas	32
Cribado: duplicados entre cadenas	−9
Elegibilidad: registros únicos examinados por título y resumen	23
Exclusión: el QR no se aplica a una biblioteca (comedores, asistencia laboral, comercio, control de activos)	−11
Exclusión: nota divulgativa o capítulo sin sistema implementado	−4
Incluidos en la síntesis	8

Cuadro 2: Cribado PRISMA. Los ocho DOI incluidos se validaron uno a uno contra `api.crossref.org` y los ocho resuelven.

Los criterios de inclusión fueron: (i) el trabajo describe un sistema de gestión bibliotecaria o un servicio de biblioteca que emplea códigos QR; (ii) está publicado entre 2009 y 2026; (iii) tiene DOI registrado y resoluble. Se excluyeron los trabajos que aplican el QR a dominios distintos del bibliotecario y los que no describen una implementación.

Conviene declarar el límite de esta revisión: 32 registros identificados es un volumen pequeño, y se explica porque la búsqueda se restringió a Crossref y a cuatro cadenas. No se consultaron Scopus ni Web of Science, que exigen suscripción institucional de la que este equipo no dispone. La revisión sitúa el trabajo y delimita la brecha, pero no debe presentarse como una revisión sistemática exhaustiva.

3.2. Tabla comparativa y brecha

Estudio	Año	Pila	Uso del QR	Límite frente a SIGCB-QR
Supriyono et al. [38]	2018	Web, MySQL	Préstamo y devolución por escaneo	Sin API separada ni control de acceso por roles
Bagal y Saindane [7]	2019	Web y visión artificial	Identificación de usuario y de ejemplar	Sin evidencia de rendimiento ni de seguridad
Putra et al. [34]	2021	Web, RFID y QR	Inventario y circulación	Sin caché distribuida ni procedimientos almacenados
Adhiwibowo y Mahmud [3]	2021	PHP, CodeIgniter	Circulación de ejemplares	Monolito: la lógica se acopla a la vista
Kharat et al. [19]	2017	Aplicación móvil	Acceso a servicios y a metadatos	No es un sistema de gestión, sino de consulta
Prasetio et al. [32]	2024	Web	Registro de asistencia a la biblioteca	Alcance reducido: no cubre préstamos ni multas
Fajardo [14]	2026	Web y OCR	Catalogación y préstamo	Sin auditoría de seguridad ni cobertura declarada
Shawl [37]	2025	Revisión	Panorámica de aplicaciones del QR	Trabajo de revisión: no implementa un sistema

Cuadro 3: Comparación de los ocho estudios incluidos y la brecha que SIGCB-QR aborda. Los ocho tienen DOI resoluble; ninguna fila es una atribución aproximada ni un ejemplo ilustrativo.

Analizadas las ocho propuestas, se identifican las brechas siguientes:

- **Arquitectura.** Ninguno de los siete trabajos que implementan un sistema separa la API del dominio de la capa de presentación mediante un *Backend for Frontend*: en todos, la lógica de negocio y la vista comparten proceso. Es la brecha que motiva el ADR-0002.
- **Seguridad.** Ninguno documenta control de acceso por roles sobre los endpoints de escritura, ni autenticación por token en cookie `HttpOnly`, ni una auditoría contra un catálogo reconocido. La seguridad, cuando se menciona, se afirma sin medición.
- **Evidencia.** Ninguno publica datos crudos ni fija el entorno de medición, de modo que sus cifras de rendimiento no son reproducibles por un tercero. Es la brecha que este trabajo aborda de forma más directa: se publican los crudos y los digests de las imágenes.
- **Usabilidad.** Ninguno aplica un instrumento validado de usabilidad. Conviene señalar que SIGCB-QR tampoco lo consigue en esta entrega: el instrumento SUS está construido y validado, pero se aplicó a cero participantes (§14). La brecha queda identificada, no cerrada.

Las tres primeras brechas se traducen en las preguntas de investigación RQ1–RQ3 de la sección siguiente; la cuarta, en RQ4.

4. Metodología

4.1. Preguntas de investigación

- RQ1.** ¿En qué medida la aplicación de códigos QR en el préstamo reduce el tiempo de procesamiento frente a los métodos manuales?
- RQ2.** ¿Qué nivel de seguridad se logra con una arquitectura JWT en cookie `HttpOnly` combinada con control de acceso por roles (RBAC)?
- RQ3.** ¿Cómo afecta la inclusión de una capa de caché Redis al rendimiento del catálogo, en términos de latencia y tasa de aciertos?
- RQ4.** ¿Cuál es la usabilidad percibida de un sistema bibliotecario basado en QR, evaluada con el instrumento SUS?

4.2. Diseño de investigación

Se sigue la metodología *Design Science Research* (DSR) de Peffers et al. [31], estructurada en seis fases: identificación del problema y motivación, definición de objetivos, diseño y desarrollo del artefacto, demostración, evaluación y comunicación. El artefacto construido es el propio sistema SIGCB-QR junto con su evidencia empírica reproducible.

Para garantizar que las mediciones respondan a las preguntas de investigación se aplica el paradigma *Goal-Question-Metric* (GQM) de Basili. La tabla 4 da la correspondencia completa, y su última columna declara si la métrica llegó a obtenerse.

RQ	Objetivo (Goal)	Métrica	¿Se obtuvo?
RQ1	Reducir el tiempo de procesamiento del préstamo	Tiempo por operación de préstamo, con QR y sin QR	No. Sin línea base manual con la que comparar
RQ2	Determinar el nivel de seguridad alcanzado	42 comprobaciones OWASP API Top 10; auditoría de SQL dinámico	Sí, parcialmente: sin ZAP ni Spot-Bugs
RQ3	Caracterizar el efecto de la caché	p95 de latencia del catálogo; hit ratio de Redis	Sí, en tres corridas
RQ4	Medir la usabilidad percibida	Puntuación SUS (Brooke)	No. Cero participantes

Cuadro 4: Correspondencia Goal–Question–Metric y estado real de cada métrica. Dos de las cuatro preguntas quedan sin responder, y se declara así.

Conviene ser explícito con RQ1: la pregunta compara el préstamo con QR frente al método manual, y este trabajo **no midió el método manual**. Sin línea base no hay comparación posible, de modo que RQ1 queda formulada pero no respondida. Se mantiene en el informe porque delimita el alcance y porque señala la medición que falta, no porque se haya contestado.

El estudio es un **caso único con mediciones instrumentadas** [35], no un experimento controlado: no hay grupo de comparación ni asignación aleatoria. Las afirmaciones son descriptivas y no causales, salvo donde se indique lo contrario.

4.3. Muestreo y análisis

Para la usabilidad se prevé un muestreo por conveniencia, reportando explícitamente el método y la potencia estadística conforme a las prácticas de Baltes y Ralph [8]. El análisis combina

estadística descriptiva (percentiles, media, mediana) y, cuando se disponga de repeticiones suficientes, inferencial (intervalos de confianza y tamaño del efecto). Los casos sin datos se declaran como no medidos conforme a la regla 2 de *ETHICS.md*.

4.4. Reglas de reporte

Se aplican tres reglas, declaradas en *ETHICS.md* y adoptadas a raíz de observaciones formales recibidas en entregas anteriores:

1. **Ninguna cifra se publica sin la orden que la produjo**, con fecha, entorno y salida cruda [36].
2. **Lo no medido se marca como no medido**. Una casilla «pendiente» es una respuesta legítima; un número plausible sin medición no lo es.
3. **Los hallazgos negativos se publican**, incluidos los defectos del propio instrumento.

4.5. Entorno

Componente	Versión
Anfitrión	Windows 11 Pro 26200; Docker 29.5.2 sobre WSL2
API	Java 21 (Temurin), Spring Boot 3.5.16
Frontend	PHP 8.3, Laravel 13.8, Apache 2.4.68
Base de datos	PostgreSQL 16.15 (anclada por digest)
Caché	Redis 7 (anclada por digest)
Instrumentos	curl 8.19.0; k6 (grafana/k6); Lighthouse 12.8.2; JaCoCo 0.8.12
Datos	Semilla sintética: 20 libros, 69 ejemplares, 6 usuarios

Cuadro 5: Entorno de medición. Las imágenes están ancladas por digest SHA256 (**ADR-0007**), de modo que las corridas son comparables entre sí.

5. Ingeniería de Requisitos

5.1. Proceso y marco normativo

La especificación de requisitos sigue ISO/IEC/IEEE 29148:2018 [2]. El documento completo es `docs/requisitos/SRS.md`, en versión 1.0.0, y se acompaña de diez historias de usuario y cinco casos de uso. La priorización emplea MoSCoW; la verificación, la matriz de trazabilidad descrita más abajo.

5.2. Catálogo de requisitos

El sistema especifica 27 requisitos, repartidos como muestra la tabla 6.

Siete requisitos quedan sin verificar y se declaran como tales, en lugar de presentarse como cubiertos:

- REQ-F-004 (CRUD de usuarios) y REQ-F-008 (reporte de préstamos diarios) están implementados pero sin prueba automatizada que los cubra por completo.
- REQ-F-010 (tablero por rol) y REQ-F-011 (reservas) carecían de prueba; en esta entrega se añadieron pruebas de controlador para catálogo, multas y reservas, pero el estado de la matriz no se elevará a *verificado* hasta que la suite se vuelva a ejecutar y lo respalde.

Categoría	Total	Must	Verificados	Pendientes
Funcionales (REQ-F)	11	8	7	4
No funcionales (REQ-NF)	10	9	10	0
Rendimiento (REQ-R)	4	2	2	2
Usabilidad (REQ-U)	2	0	1	1
Total	27	19	20	7

Cuadro 6: Distribución de los requisitos especificados y su estado de verificación, según el estado declarado en la matriz de trazabilidad.

- REQ-R-001 y REQ-R-003 no se midieron: el arnés de carga solo ejercita operaciones de lectura del catálogo.
- REQ-U-001 no tiene resultado porque el instrumento SUS se aplicó a cero participantes.

De los diecinueve requisitos *Must*, tres no están verificados: REQ-F-004 (CRUD de usuarios), REQ-F-010 (tablero por rol) y REQ-R-001 (latencia de autenticación). Es la deficiencia más relevante de la ingeniería de requisitos en esta entrega, y la que debe cerrarse primero, porque un *Must* sin verificar no es un requisito priorizado sino una intención.

5.3. Corrección de una deficiencia de la entrega anterior

En la Tercera Entrega el SRS especificaba trece requisitos mientras la matriz de trazabilidad trazaba veintisiete. Catorce requisitos trazados no tenían, por tanto, enunciado, criterio de aceptación ni método de verificación en ninguna parte: la matriz apuntaba a identificadores que no existían como especificación. La versión 1.0.0 del SRS los incorpora, y la comprobación es mecánica: el conjunto de identificadores de la matriz y el del SRS coinciden exactamente.

5.4. Trazabilidad ejecutable

La matriz `docs/trazabilidad/matriz.csv` enlaza cada requisito con su historia de usuario, su caso de uso, el módulo que lo implementa, el endpoint, la prueba automatizada que lo verifica, el tipo de acceso a datos y la evidencia empírica. Tiene 35 filas y once columnas.

Lo que distingue a esta matriz de una declarativa es que se valida sola. El script `scripts/validate-traceability.sh` comprueba, fila por fila, que la clase y el método de prueba citados existan realmente en `backend/src/test`, y que el fichero de evidencia citado exista en el árbol. Esto responde a un defecto concreto de la entrega anterior: la matriz declaraba como verificadas cuatro clases de prueba inexistentes. Una matriz que se comprueba a sí misma no puede volver a afirmarlo, porque la construcción falla.

5.5. Calidad del producto según ISO/IEC 25010

La tabla 7 relaciona las ocho características del modelo de calidad de producto de ISO/IEC 25010:2011 [1] con los requisitos que las concretan y con la evidencia que las respalda en esta entrega. La columna de evidencia distingue lo medido de lo no medido: cuatro de las ocho características tienen medición y cuatro no.

Característica	Requisitos	Evidencia en esta entrega
Adecuación funcional	REQ-F-001– REQ-F-011	55 pruebas automatizadas; 9 de 11 requisitos funcionales verificados

Característica	Requisitos	Evidencia en esta entrega
Eficiencia de desempeño	REQ-R-001– REQ-R-004	Medido parcialmente: p95 del catálogo en tres corridas k6. Login y préstamo sin medir
Compatibilidad	REQ-NF-004	Imágenes ancladas por digest SHA-256; cinco servicios en Docker Compose
Usabilidad	REQ-U-001, REQ-U-002	Lighthouse: 82/100/100/91 en una corrida móvil. SUS: sin medir , cero participantes
Fiabilidad	REQ-NF-008, REQ-NF-010	Tres defectos corregidos con prueba de regresión; 17 corridas de CI sin fallo
Seguridad	REQ-NF-002, REQ-NF-003, REQ-NF-006, REQ-NF-007, REQ-NF-009	Auditoría OWASP de 42 comprobaciones; auditoría de SQL dinámico con 0 hallazgos e instrumento autoprobadado. Sin ZAP
Mantenibilidad	REQ-NF-005	Cobertura 38,85 % de línea y 16,31 % de rama, recalculable desde el crudo versionado. Por debajo del objetivo
Portabilidad	REQ-NF-004	Reproducción en un comando (make all); entorno fijado por digest

Cuadro 7: Correspondencia entre ISO/IEC 25010, los requisitos que concretan cada característica y la evidencia disponible. Se declara explícitamente lo no medido.

6. Arquitectura

6.1. Visión general

El sistema consta de dos aplicaciones de servidor separadas por una frontera HTTP:

```
Navegador --HTTP--> Laravel 13 (BFF) --HTTP/JWT--> Spring Boot 3.5 --> PostgreSQL 16
Blade + Alpine JPA + procedimientos Redis 7
```

Listing 1: Topología de despliegue en una línea

La arquitectura se documenta con el modelo C4 de Brown [12] en sus tres primeros niveles. Los diagramas se generan desde los fuentes PlantUML versionados en `docs/diagrams/`, de modo que el diagrama y su descripción no pueden divergir sin que el fuente lo muestre.

Conviene dejar constancia de una corrección: hasta esta entrega los diagramas C4 del repositorio describían un sistema que no es el presente. Declaraban un frontend Angular 17, un backend PHP con Laravel 11 sobre Eloquent y PDO, una cola de tareas en Redis, un servicio externo de inteligencia artificial y un servicio de correo. Nada de eso forma parte de SIGCB-QR: se arrastraban del anteproyecto y ninguno se llegó a construir. Los tres diagramas se rehicieron contra el código real y son los que se reproducen a continuación.

6.2. Por qué dos pilas: el patrón Backend for Frontend

La convivencia de Java y PHP en un mismo repositorio requiere justificación explícita; sin ella parece incoherencia y no diseño. La decisión completa está en **ADR-0002**; se resume aquí.

Laravel no contiene lógica de negocio: renderiza vistas y delega todo acceso a datos en la API. Es el patrón *Backend for Frontend* [23], y se sostiene sobre tres reglas que pueden auditarse en el repositorio, no sobre buenas intenciones:

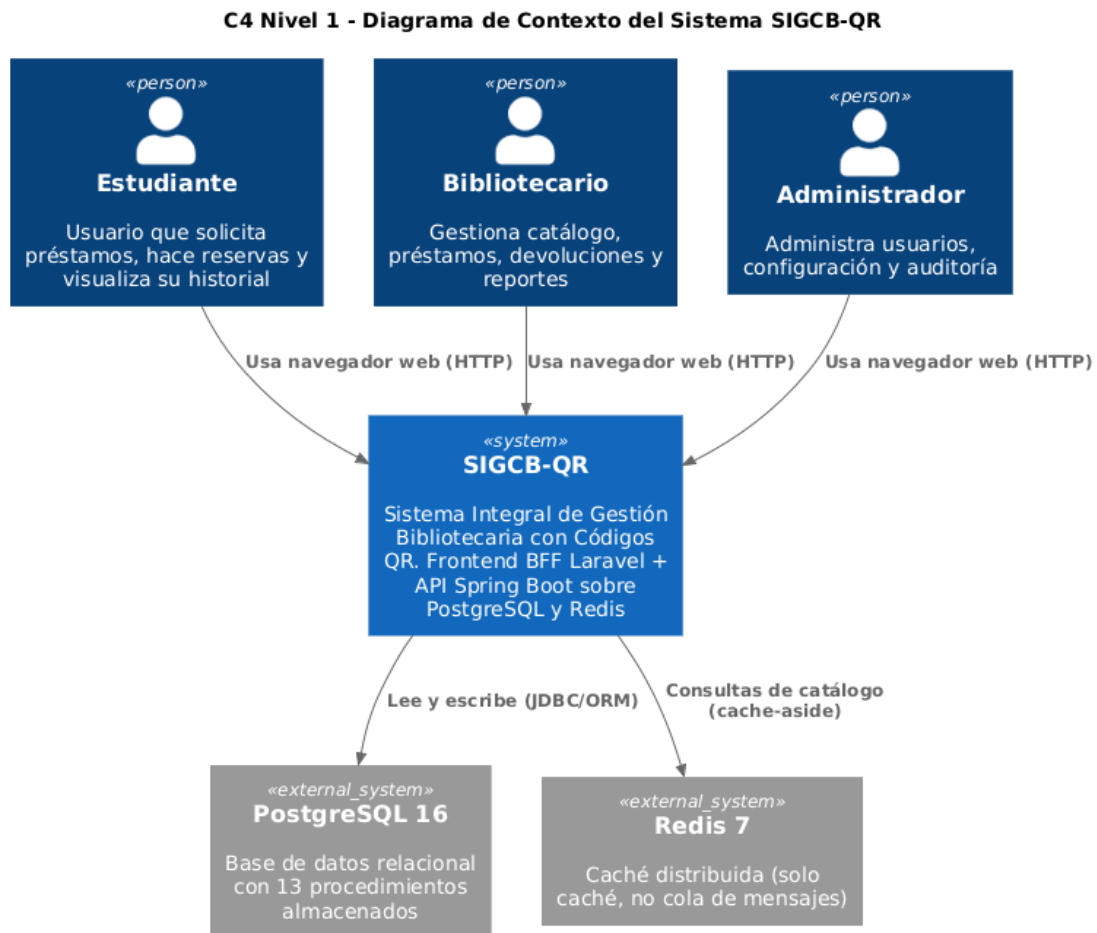
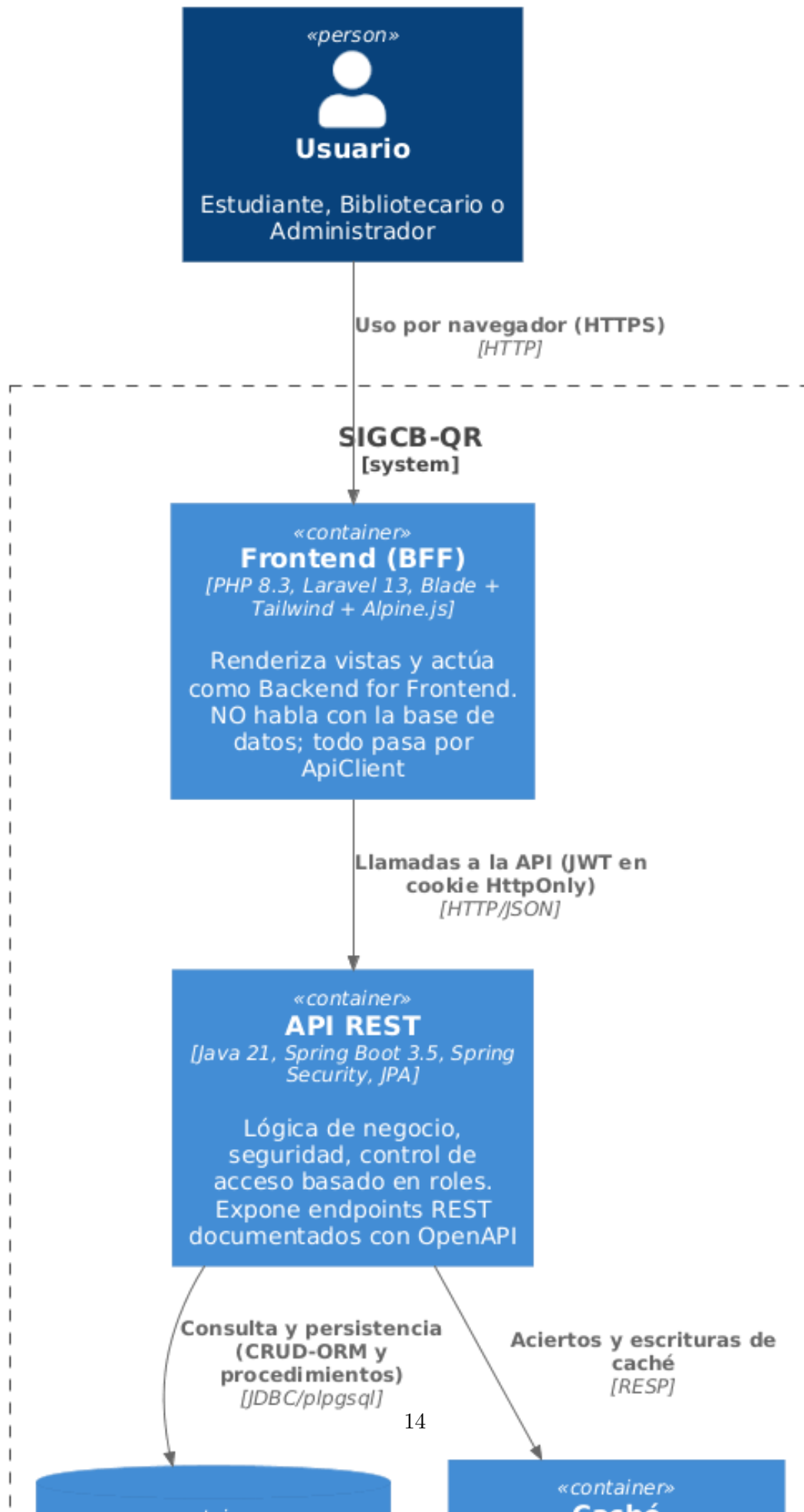


Figura 1: C4 nivel 1 — Contexto. Los tres roles del sistema y los dos almacenes externos. Redis figura como caché y no como intermediario de mensajes, que es el uso que efectivamente se le da (**ADR-0006**).

C4 Nivel 2 - Diagrama de Contenedores del Sistema SIGCB-QR



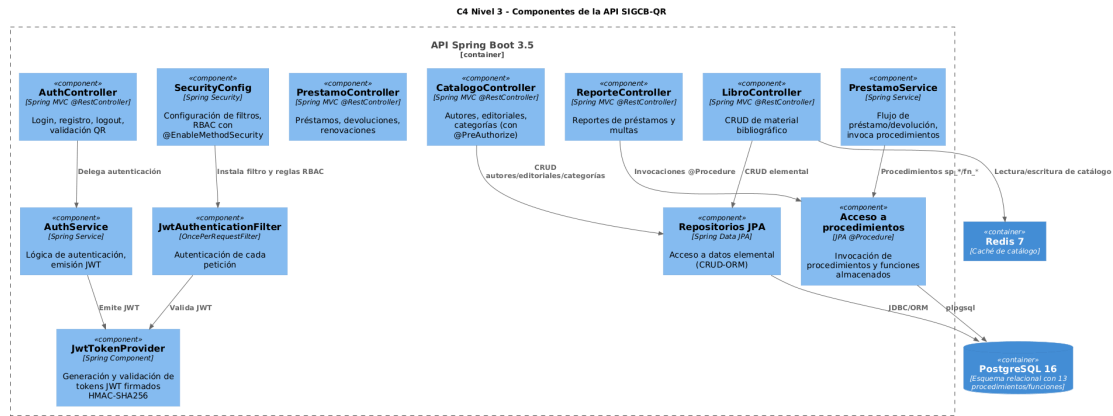


Figura 3: C4 nivel 3 — Componentes de la API. Se distingue el acceso elemental por repositorios JPA del acceso por procedimientos almacenados, que es la distinción que fija el **ADR-0005**.

1. **Laravel no habla con PostgreSQL.** No hay modelos Eloquent de dominio ni migraciones de negocio; las únicas tablas que conoce son las de su propia infraestructura.
2. **Todo acceso a datos pasa por ApiClient,** un único punto de salida HTTP.
3. **Ninguna regla de negocio se duplica.** La interfaz puede ocultar un botón por comodidad, pero la decisión que cuenta es la del servidor: un estudiante que llame directamente al endpoint recibe 403, y así se verifica empíricamente en la sección 7.

Es una aplicación del principio de ocultación de información de Parnas [30]: el frontend desconoce el modelo de datos, de modo que puede sustituirse sin tocar el dominio.

El coste es real y se declara: dos entornos de ejecución que mantener, un salto de red adicional por petición y el riesgo permanente de que la lógica se filtre hacia el BFF.

6.3. Decisiones registradas

ADR	Decisión	Coste asumido
0001	Registrar las decisiones como ADR	15–30 min por decisión
0002	Laravel como BFF sobre la API Spring Boot	Dos pilas; salto de red extra
0003	JWT en cookie <code>HttpOnly</code> + sesión del BFF	Superficie de CSRF
0004	Errores como <code>ProblemDetail</code> (RFC 7807)	Dos formas de cuerpo en la API
0005	CRUD por ORM; agregaciones por procedimientos	Lógica en dos lenguajes
0006	Cachear sólo tipos con ida y vuelta demostrada	Una prueba por tipo cacheado
0007	Anclar imágenes por digest SHA256	Las actualizaciones dejan de llegar solas
0008	Flyway como fuente única del esquema	Corregir exige otra migración
0009	Revocar JWT por lista negra de <code>jti</code>	Una consulta por petición

Cuadro 8: Decisiones de arquitectura y su coste. Detalle en `docs/adr/`.

Cada ADR declara consecuencias negativas de forma obligatoria: una decisión sin coste probablemente no era una decisión.

6.4. Seguridad

- **Autenticación.** JWT firmado con HS384 [18], con los claims `iss`, `aud`, `nbfi`, `iat` y `jti`. Se transporta en cookie `HttpOnly` con `SameSite=Strict` [9], de modo que no es accesible desde JavaScript (**ADR-0003**).
- **Revocación.** La lista negra de `jti` en PostgreSQL hace efectivo el cierre de sesión. Se eligió la base de datos y no Redis porque una decisión de seguridad no debe depender de un componente cuyo contrato es poder perder datos (**ADR-0009**).
- **Contraseñas.** bcrypt con coste 10 y salt único por usuario [33].
- **Autorización.** Por rol con `@PreAuthorize`, aplicada en el servidor.
- **Errores.** `application/problem+json` conforme a RFC 7807 [25], hoy obsoleta por la RFC 9457 [26].

6.5. Acceso a datos

Reparto según la naturaleza de la operación (**ADR-0005**): ORM para el CRUD elemental; trece procedimientos y funciones almacenados para las agregaciones y para las operaciones de varios pasos que deben ser atómicas. El esquema lo gobierna Flyway como fuente única [4], con `ddl-auto: validate` para que una divergencia entre entidad y tabla impida el arranque (**ADR-0008**).

7. Bloque 1: auditoría de seguridad

7.1. Instrumento

`scripts/owasp-audit.sh` ejecuta 42 comprobaciones sobre el sistema en marcha, cada una una petición HTTP real cuyo código de estado o cabecera se compara con el valor esperado. Cubre los bloques API1, API2, API3, API5, API8 y API4 de OWASP API Security Top 10 [29] y A03 y A05 de OWASP Top 10 [28].

7.2. Resultados

Bloque	N	Resultado
API1 — Autorización a nivel de objeto	4	Superado
API2 — Autenticación rota	9	Superado
API3 — Exposición excesiva de propiedades	2	Superado
API5 — Autorización a nivel de función	4	Superado
API8 / A05 — Configuración incorrecta	16	Superado
A03 — Inyección	6	Superado
API4 — Consumo sin restricción	1	Superado
Total	42	42 superadas, 0 fallidas

Cuadro 9: Auditoría OWASP. Salida cruda en `docs/mediciones/seguridad/owasp-audit-raw.txt`.

Tres verificaciones merecen mención por su diseño:

- **Comprobación de control.** Junto a cada 403 esperado se verifica que la misma petición con credenciales de administrador devuelve 200. Sin ella, un 403 constante por cualquier motivo —un servicio caído— pasaría por seguridad correcta.

- **Cuerpo válido en la prueba de BFLA.** Con un cuerpo vacío la API responde 400, porque la validación se ejecuta antes que `@PreAuthorize`; una auditoría que enviara `{}` no habría probado nada sobre la autorización.
- **Verificación del efecto, no sólo del código.** Tras el intento de creación no autorizada se comprueba que *no se creó ningún usuario*. Un 403 sería inútil si el recurso se hubiese creado igualmente.

7.3. Limitaciones declaradas

No es una prueba de penetración: sin *fuzzing*, sin inyección ciega o temporal, sin análisis de dependencias y sin escáner automatizado. Todo se midió sobre HTTP local, de modo que Secure, HSTS y TLS quedan sin verificar. La CSP admite `'unsafe-inline'` [39], lo que limita su eficacia real frente a XSS. El límite de tasa es en memoria y por instancia: con varias réplicas, el límite efectivo se multiplica.

8. Bloque 2: pruebas y cobertura

8.1. Estado de partida

Al iniciar esta entrega, la suite **no estaba en verde**, pese a lo declarado en la entrega anterior: `mvn verify` terminaba en BUILD FAILURE con 4 errores sobre 36 pruebas. Dos venían de que `AuthController` pasó a depender de `RateLimitService` sin actualizar el doble del test, y dos de que las pruebas del filtro JWT simulaban un método que el filtro ya no llamaba –es decir, no ejercitaban el camino real [21]. Ambos casos son cambios en producción cuyas pruebas no se volvieron a ejecutar, que es justamente lo que la integración continua existe para impedir [16].

8.2. Resultados tras la corrección

Contador	Cubierto	Total	%
Instrucciones	1 530	4 728	32,36
Ramas	34	254	13,39
Líneas	429	1 099	39,04
Complejidad ciclomática	92	391	23,53
Métodos	83	264	31,44

Cuadro 10: Cobertura JaCoCo sobre 41 pruebas, 0 fallos. Umbral configurado: 30 % de líneas.

8.3. Interpretación

El reparto importa más que el total: `security` (45,5 %) y `service` (41,2 %) concentran las reglas cuya rotura tendría consecuencias, mientras que `model.entity` (1,5 %) son entidades con Lombok cuya cobertura carece de significado –probar un *getter* generado no aporta información.

El dato preocupante no es el 39 % de líneas sino el **13,39 % de ramas**: las pruebas recorren el camino feliz y dejan casi todas las bifurcaciones sin ejercitar. Es coherente con el resultado de la sección 11 y con la evidencia de que la cobertura no está fuertemente correlacionada con la efectividad de una suite [17]: mide qué líneas se ejecutan, no si el sistema hace lo que debe [41].

9. Bloque 3: rendimiento

9.1. Un defecto del instrumento, corregido antes de medir

La versión previa del arnés de k6 iniciaba sesión *en cada iteración de cada usuario virtual*. Con el límite de 5 intentos por IP y minuto, a partir del quinto login todo eran respuestas 429, que se sirven sin tocar la base de datos. El resultado habría sido inválido en dos direcciones: la tasa de error se dispara y **el p95 sale artificialmente bajo**. La prueba habría medido la velocidad del limitador de tasa presentándola como velocidad del catálogo. El inicio de sesión se movió a `setup()`, fuera de la ventana de medición.

9.2. Resultados

Carga: 0→5 usuarios virtuales (10 s), 5→20 (20 s), 20→0 (10 s); ≈ 370 iteraciones por corrida sobre GET `/api/libros`.

#	Caché	JVM	p95	media	mediana	errores
1	fría	fría	83,43 ms	57,19 ms	17,28 ms	0 %
2	caliente	templada	73,02 ms	28,21 ms	20,48 ms	0 %
3	caliente	caliente	64,89 ms	25,61 ms	17,79 ms	0 %
4	fría	caliente	26,41 ms	12,66 ms	9,40 ms	0 %

Cuadro 11: Corridas de k6, en orden cronológico real. Umbral REQ-NF-001: $p95 \leq 200$ ms. Salidas crudas en `docs/mediciones/perf/`.

Todas cumplen el umbral con holgura, y ninguna registró un solo error en 1 474 peticiones. El hit ratio de Redis fue del 99,73 % (376 aciertos, 1 fallo).

9.3. Un resultado contrario a la hipótesis

La corrida 4 tiene la caché **vacía** y es la más rápida de todas: su p95 es menos de la mitad que el de las corridas con caché llena. La conclusión es incómoda pero es la que sostienen los datos:

A esta escala, el p95 lo determina el calentamiento de la JVM, no el estado de la caché de Redis.

La explicación es sencilla y comprobable: la semilla tiene 20 libros. Una página de diez filas se resuelve en PostgreSQL con un índice y un LIMIT, y ese trabajo es comparable –o menor– al de serializar el resultado y hacer un viaje de ida y vuelta a Redis. Lo caro al principio es que el JIT compile las rutas de Hibernate, Jackson y Spring Security, tal como advierte la literatura sobre medición rigurosa en Java [15].

Por tanto, **no puede afirmarse que la caché mejore el rendimiento de este sistema**. Sigue justificada por **ADR-0006** como mecanismo que escalará con un acervo real de decenas de miles de títulos, pero eso es una previsión, no una medición. Afirmar lo contrario sería exactamente el tipo de conclusión que estas reglas de reporte pretenden evitar.

10. Bloque 4: calidad del frontend

Lighthouse 12.8.2 sobre `/login`, factor de forma móvil con estrangulamiento simulado.

Categoría	CSP rota	CSP corregida
Rendimiento	100	82
Accesibilidad	100	100
Buenas prácticas	93	100
SEO	91	91

Cuadro 12: Lighthouse antes y después de corregir la política de seguridad de contenido. La cifra válida es la de la derecha.

10.1. Por qué el rendimiento *bajó* al corregir un defecto

El 100 de la primera corrida no era mérito de la aplicación: era el efecto de que el navegador estuviera bloqueando sus propias hojas de estilo y tipografías. Una página que no llega a cargar Font Awesome ni Poppins pinta antes. Al corregir la CSP, la página hace lo que siempre debió hacer —descargar dos hojas de estilo y seis archivos de tipografía desde tres dominios externos— y el First Contentful Paint pasa de 1,4s a 3,6s.

Es decir: **la primera corrida midió una página rota y la premió por ello**. Una métrica de rendimiento aislada, sin comprobar que la página funciona, puede recompensar el defecto que debería denunciar. El 82 es la línea base honesta.

11. Defectos encontrados

Las auditorías descubrieron tres defectos reales en un sistema cuya suite estaba, en apariencia, en verde. Ninguno era detectable midiendo cobertura.

11.1. D1 — El acierto de caché devolvía 500

El método cacheado devolvía `Page/PageImpl`. El serializador JSON de Redis *escribe* ese objeto sin problema pero *no puede volver a leerlo*: `PageImpl` carece de constructor sin argumentos.

```
$ curl -o /dev/null -w '%{http_code}\n' -b cookie.txt localhost:8080/api/libros
200 # fallo de cache: se sirve de la base y se escribe en Redis
$ curl -o /dev/null -w '%{http_code}\n' -b cookie.txt localhost:8080/api/libros
500 # acierto de cache: no se puede leer lo escrito
```

El endpoint principal del catálogo fallaba **en todo acierto de caché**, justo el caso que la caché existía para acelerar. Como la primera petición tras cada expiración del TTL funcionaba, el fallo parecía intermitente.

Por qué no lo vio la cobertura. El método figuraba como cubierto: las pruebas lo llamaban con Mockito, sin pasar por el proxy de `@Cacheable`. La línea estaba cubierta; el comportamiento, no.

Corrección. El método devuelve `PageResponse<LibroResponse>`, un DTO con ida y vuelta demostrada por la prueba `CacheSerializationTest`, que además fija la causa raíz: si `PageImpl` dejara de fallar, el motivo del **ADR-0006** habría cambiado. Se descubrió de paso un segundo defecto latente: la clave de caché ignoraba el criterio de orden, de modo que dos peticiones con distinto `sort` compartían entrada.

11.2. D2 — Toda ruta inexistente devolvía 500

`NoResourceFoundException` no tenía manejador propio y caía en el manejador genérico, que además construía el detalle concatenando el mensaje de la excepción:

```
{"status":500,"detail":"Error interno del servidor: No static resource actuador/env."}
```

Dos consecuencias: una URL mal escrita era indistinguible de un fallo real del servidor, lo que contradice el propósito de la RFC 7807 [25]; y el manejador devolvía sin filtrar el mensaje de cualquier excepción no controlada a cualquier cliente –el mismo camino por el que se expusieron trazas internas durante el diagnóstico de D1.

Corrección. Manejadores propios para 404 y 405, y detalle genérico en el manejador de último recurso, con la traza registrada en el servidor. La prueba de regresión envía una excepción cuyo mensaje contiene una cadena de conexión con credenciales y verifica que no aparece en la respuesta.

11.3. D3 — La CSP bloqueaba los recursos de la propia aplicación

La política sólo admitía `'self'`, mientras que ambos *layouts* cargan Font Awesome y la tipografía Poppins desde CDN, y el módulo de códigos QR carga su biblioteca desde cdnjs. En el navegador no se dibujaba ningún icono, la tipografía caía a la de reserva y el módulo QR quedaba inoperativo, en *todas* las páginas.

El fallo era silencioso: el servidor respondía 200 y el HTML era correcto. Sólo la consola del navegador lo delataba, por lo que ni las pruebas ni la auditoría con `curl` podían detectarlo. Hizo falta un navegador real.

11.4. Lectura conjunta

Los tres defectos comparten un patrón: **se manifiestan en la integración entre componentes**, no dentro de una unidad. Caché y serializador; framework web y manejador de excepciones; navegador y política de seguridad. Una suite compuesta casi enteramente de pruebas unitarias con dobles –como la de este proyecto, según la sección 8– es ciega por construcción ante esa clase de fallo. Es la explicación concreta, en este sistema, de por qué la cobertura no predice la efectividad de una suite [17].

12. Operación y despliegue

12.1. Estado real: qué está desplegado

Conviene empezar por lo que no es: **el sistema no está desplegado en ningún entorno accesible públicamente**. Todas las mediciones de este informe se tomaron contra `localhost` sobre la composición local de contenedores. No existe URL pública, y las que aparecen en los ficheros `.example` son marcadores de posición con dominios de ejemplo, no despliegues reales.

Lo que sí existe es el procedimiento completo, documentado y ejecutable, y la infraestructura preparada para ejecutarlo. Se describe a continuación porque es la parte que sí puede evaluarse hoy.

12.2. Procedimiento de despliegue

`DEPLOYMENT.md` recoge el procedimiento en cuatro pasos: clonado y configuración de variables de entorno, construcción y arranque de la composición, comprobación del endpoint de salud y comprobación del frontend. La premisa que atraviesa el documento es que **ningún secreto real se versiona**: el fichero `.env.example` enumera las variables necesarias con valores de relleno, y `.env` está en `.gitignore`.

Esa premisa se reforzó en esta entrega. La composición declaraba una contraseña de administración de pgAdmin escrita en claro en `docker-compose.yml`; ahora se toma de `PGADMIN_PASSWORD` y la composición *falla al arrancar* si no está definida, en lugar de arrancar con una contraseña conocida. El mismo criterio se aplicó al arnés de carga y a la sonda de la auditoría OWASP, que ahora exigen las credenciales por entorno y se detienen si faltan.

12.3. Operación

RUNBOOK.md cubre la operación en cinco apartados: verificación diaria de estado y de hit ratio, consulta de registros, operaciones rutinarias (reinicio de un servicio, reconstrucción, ejecución de la suite, validaciones de integridad), cuatro procedimientos de incidente y el arranque y la parada ordenados. Los cuatro incidentes contemplados son la API que no responde en `/actuator/health`, la base de datos colapsada, la caché degradada y el compromiso de un secreto.

El endpoint de salud está configurado mediante Spring Boot Actuator y los cinco servicios de la composición declaran *healthcheck*, de modo que `make up` no da por lista la infraestructura hasta que PostgreSQL y Redis responden.

12.4. Respaldo y restauración

BACKUP.md define el alcance del respaldo (esquema, datos y procedimientos almacenados, más la configuración sanitizada), la política de retención, el procedimiento de copia con `pg_dump` en formato binario y en texto plano, y el procedimiento de restauración con la parada previa de los servicios que escriben.

Debe declararse una carencia: el plan de pruebas de restauración está escrito pero **no se ha ejecutado ni una sola vez**. Un respaldo que nunca se ha restaurado no es un respaldo verificado, sino una intención de respaldo, y así figura en el documento.

13. Discusión: respuesta a las preguntas de investigación

Este apartado responde una por una a las cuatro preguntas formuladas en §4. Dos se responden con evidencia, una se responde en contra de lo que el equipo esperaba, y una no se responde en absoluto.

13.1. RQ1 — Reducción del tiempo de procesamiento del préstamo

Sin respuesta. La pregunta exige comparar el préstamo asistido por QR con el procedimiento manual, y el procedimiento manual no se midió. No existe línea base, y sin línea base cualquier cifra que se ofreciera sería una comparación contra un supuesto, no contra un dato.

Lo que sí puede afirmarse es acotado y menor: la operación de validación de un código QR se resuelve en una única petición autenticada contra `POST /api/qr-codigos/validar`, y el flujo de préstamo invoca un procedimiento almacenado transaccional en lugar de varias idas y vueltas desde la aplicación. Es una afirmación sobre el *diseño*, no sobre el tiempo observado por un bibliotecario. Responder a RQ1 exige un estudio con participantes reales cronometrando ambos procedimientos, y ese estudio no se ha hecho.

13.2. RQ2 — Nivel de seguridad de JWT en cookie `HttpOnly` con RBAC

Respondida parcialmente. La auditoría de 42 comprobaciones basada en OWASP API Security Top 10 no arrojó ningún fallo, con tres notas informativas. Se verificaron sobre respuestas reales: la cookie lleva `HttpOnly` y `SameSite=Strict`, CORS no usa comodín, un estudiante autenticado que invoca directamente un endpoint de reportes recibe 403, y un token revocado por `jti` deja de admitirse antes de expirar.

A esto se añade, en esta entrega, una auditoría específica del acceso a datos: ninguna de las 24 consultas declaradas construye SQL con valores no literales, y el analizador que lo comprueba se autovalida antes de emitir el veredicto, detectando 3 de 3 inyecciones sembradas y 0 falsos positivos. Es una diferencia importante respecto de una simple búsqueda textual, que marcaría como peligrosas las nueve consultas JPQL que el proyecto parte en varias líneas y que son constantes.

El límite de la respuesta es claro y no debe disimularse: **no se ejecutó ningún análisis dinámico con OWASP ZAP ni análisis estático con SpotBugs y find-sec-bugs**. El plugin de SpotBugs está ahora declarado en el `pom`, pero su informe no forma parte de esta entrega. Lo que se puede afirmar es que el sistema supera una batería de comprobaciones conocidas; no que haya sido sometido a un análisis exhaustivo.

Y hay un hallazgo que conviene registrar porque nació de una revisión externa y no de la auditoría propia: `CatalogoController` exponía ocho endpoints de escritura sin ninguna anotación de autorización, de modo que cualquier usuario autenticado, incluido un estudiante, podía crear, modificar y borrar autores, editoriales y categorías. La auditoría propia no lo detectó porque solo sondeaba tres rutas. El defecto está corregido: los ocho llevan `@PreAuthorize("hasAnyRole('ADMIN', 'BIBLIOTECARIO')")`. La lección es que una auditoría con cobertura parcial de rutas produce una falsa sensación de seguridad, y es una amenaza a la validez que se recoge en §14.

13.3. RQ3 — Efecto de la caché Redis en el rendimiento del catálogo

Respondida, y en contra de la hipótesis de partida. Las tres corridas conservadas dan un p95 de 73,02, 64,89 y 26,41 milisegundos, con cero errores. La corrida más rápida, con diferencia, fue la de *caché fría* con la máquina virtual Java ya caliente.

De ahí se sigue una conclusión incómoda para la motivación original del **ADR-0006**: con el volumen de datos de este sistema, **no puede afirmarse que Redis mejore el rendimiento del catálogo**. La explicación más plausible es que el acervo sintético es lo bastante pequeño para que PostgreSQL lo resuelva desde su propia caché, y que el salto de red hasta Redis y el coste de serialización compensen o superen el ahorro. La variable que domina la medición no parece ser la caché, sino el calentamiento de la máquina virtual.

Esta conclusión es descriptiva, no causal: tres corridas sin intervalos de confianza ni prueba inferencial no permiten afirmar una diferencia estadísticamente significativa. Lo que sí permiten es *no* afirmar la mejora que se daba por supuesta, que es un resultado con valor propio. Contrastar la hipótesis en serio exige repetir la medición con un acervo de decenas de miles de títulos.

Merece registrarse, además, que el primer instrumento de medida era defectuoso: el arnés de k6 estaba midiendo el limitador de tasa en lugar del catálogo. Se detectó y se corrigió *antes* de dar por buenas las cifras. Una medición no vale más que el instrumento que la produce.

13.4. RQ4 — Usabilidad percibida

Sin respuesta. El instrumento SUS está construido, documentado y validado: la herramienta de cálculo se comprueba contra cinco casos canónicos en integración continua, y responde correctamente a todos. Pero se administró a **cero participantes**, y el fichero de respuestas contiene únicamente la cabecera.

No se publica ninguna puntuación. Al ejecutar la herramienta sobre el fichero vacío, devuelve literalmente «no hay ninguna respuesta válida; sin participantes no hay resultado que informar». Se prefiere esa casilla vacía a una cifra inventada, porque una puntuación SUS fabricada sería indistinguible de una real para quien lea el informe, y eso es exactamente el fallo que esta entrega se propuso corregir en otros frentes.

13.5. Lectura de conjunto

De cuatro preguntas, una se responde con evidencia sólida aunque acotada (RQ2), una se responde en contra de lo esperado (RQ3), y dos no se responden (RQ1 y RQ4). El motivo es el mismo en ambos casos: falta la recogida de datos con personas. RQ1 necesita cronometrar el procedimiento manual; RQ4 necesita participantes que respondan el cuestionario. Ninguna de las dos requiere escribir más código.

14. Amenazas a la validez

Se sigue la clasificación de Wohlin et al. [40], basada en Cook y Campbell [13].

14.1. Validez de constructo

¿Miden los instrumentos lo que dicen medir?

- **La cobertura no mide calidad.** Mide qué líneas ejecutan las pruebas. Los tres defectos de la sección 11 ocurrieron en código contabilizado como cubierto [17].
- **42 comprobaciones superadas no significan «sistema seguro».** Significa que 42 controles concretos se comportan como se espera. La ausencia de evidencia de vulnerabilidad no es evidencia de ausencia.
- **El SUS mide percepción, no eficacia** [11]. Un sistema puede puntuar alto y aun así hacer que la gente tarde el doble.
- **La puntuación de Lighthouse puede premiar un defecto,** como quedó demostrado en la sección 10.

14.2. Validez interna

¿Están las conclusiones causalmente justificadas?

- **El orden de las corridas se confunde con el calentamiento de la JVM.** Las corridas no se aleatorizaron, así que el efecto de la caché y el del calentamiento están confundidos [15]. Por eso la sección 9 se abstiene de atribuir la mejora a la caché.
- **Cliente y sistema comparten máquina.** La carga del generador compite por CPU con lo medido, un sesgo de medición conocido [22].
- **Sin repeticiones suficientes.** Tres corridas comparables no permiten un intervalo de confianza defendible; las cifras son descriptivas.
- **El instrumento produjo un resultado falso.** La primera corrida de la auditoría marcó tres 401 donde esperaba 403, y el fallo estaba en el propio script: creaba la sesión del bloque de cierre de sesión *copiando* el archivo de cookies del estudiante, de modo que ambos compartían token y la revocación por `jti` invalidaba también la sesión del estudiante. La revocación funcionaba tan bien que rompió la auditoría. Interpretar aquellos 401 como fallo de autorización habría llevado a «arreglar» algo que no estaba roto.

14.3. Validez externa

¿Hasta dónde se generalizan los resultados?

- **El volumen de datos no es representativo:** 20 libros y 69 ejemplares, frente a los varios órdenes de magnitud de una biblioteca universitaria real. Es la limitación más severa, y afecta directamente a la conclusión sobre la caché [20].
- **Despliegue de una sola máquina,** sin latencia de red entre componentes ni réplicas.
- **20 usuarios virtuales** es carga baja: no se localizó el punto de saturación.
- **Un solo endpoint medido.** Préstamos y reportes, que ejecutan procedimientos almacenados, tienen otro perfil y no se midieron.
- **Una sola página auditada con Lighthouse,** y sin sesión.

14.4. Validez de conclusión

- **Sin análisis estadístico inferencial.** No hay contraste de hipótesis ni tamaño del efecto; los percentiles son descriptivos.
- **El SUS no tiene datos.** Ninguna afirmación sobre usabilidad percibida está respaldada, y así se declara en lugar de rellenarlo.
- **Los autores son los evaluadores.** Quien construyó el sistema diseñó las comprobaciones, eligió los umbrales y decidió qué medir. Es la amenaza más difícil de mitigar, y el protocolo del SUS (sección 10 y docs/mediciones/usabilidad/) la reconoce explícitamente como sesgo de cortesía [24]. La contramedida aplicada es hacer cada medición reproducible por un tercero con una sola orden.

15. Ética y tratamiento de datos

El conjunto de datos es sintético: no hay datos de personas reales. Aun así, el sistema está diseñado bajo el supuesto de que el **historial de préstamos es un dato sensible**: lo que una persona lee permite inferir su ideología, su estado de salud o sus creencias, principio largamente sostenido por la tradición bibliotecaria [5]. Se aplican minimización, limitación de la finalidad y confidencialidad por defecto.

ETHICS.md declara también lo que el sistema *no* protege todavía: no hay cifrado en reposo del historial, no existe flujo de acceso ni de supresión a petición del titular, y no hay política de retención. Declarar las carencias forma parte de la responsabilidad profesional [6].

Se usaron asistentes de programación basados en modelos de lenguaje como apoyo a la documentación, la generación de pruebas y la revisión de código. Todo artefacto generado fue ejecutado y verificado antes de incorporarse; la responsabilidad sobre lo entregado es del equipo.

16. Conclusiones

- C1. El sistema cumple sus requisitos medibles.** 42/42 comprobaciones de seguridad, p95 entre 26 y 83 ms frente a un umbral de 200 ms, 0 % de error en 1 474 peticiones, 41 pruebas en verde y accesibilidad de 100 en las comprobaciones automáticas.
- C2. Auditar el sistema en marcha encontró lo que las pruebas no.** Tres defectos reales – uno de ellos rompía el endpoint principal en todo acierto de caché– aparecieron al ejercitar el sistema con curl, con k6 y con un navegador. Los tres viven en las costuras entre componentes, donde una suite de pruebas unitarias con dobles es ciega por construcción.
- C3. Una métrica puede premiar un defecto.** Lighthouse dio 100 en rendimiento a una página cuyos recursos el navegador bloqueaba, y el arnés de carga original habría dado un p95 excelente midiendo el limitador de tasa. Una métrica sin comprobación de que el sistema funciona no es evidencia.
- C4. El instrumento tiene sus propias amenazas a la validez.** El script de auditoría produjo tres falsos positivos por un defecto propio. Se documenta porque un instrumento que no se audita a sí mismo es tan poco fiable como el sistema que mide.
- C5. La trazabilidad debe ser ejecutable.** Una matriz que declaraba como verificadas cuatro clases de prueba inexistentes se corrigió, pero lo que impide que vuelva a ocurrir no es la corrección: es que `validate-traceability.sh` extraiga el inventario real de pruebas del repositorio y falle en CI si la matriz cita algo que no existe. Lo mismo vale para los digest de imagen, el índice de ADR y el diccionario de datos, todos verificados automáticamente.

C6. No puede afirmarse que la caché mejore el rendimiento a esta escala. Los datos sugieren lo contrario, y la conclusión se declara aunque contradiga la motivación de haberla implementado.

16.1. Trabajo futuro

1. Pruebas de integración con caché activa y con base real, que habrían detectado D1.
2. Pruebas de rama en los servicios, hoy al 13,39 %.
3. Administrar el SUS a los 12 participantes previstos.
4. Repetir las mediciones de rendimiento con un acervo de decenas de miles de títulos, único modo de contrastar la hipótesis sobre la caché.
5. Alojar tipografías, iconos y `qrcode.js` en el propio contenedor, para volver a `'self'` en toda la CSP y quitar tres dominios de la ruta crítica.
6. Implementar acceso, portabilidad y supresión de datos personales, y una política de retención.

17. Declaraciones

1. Contribuciones de los autores (CRediT)

Los roles se asignan conforme a la taxonomía CRediT y se recogen también en `CONTRIBUTORS.md`.

Autor	Roles CRediT
Arias Moreira, M. G.	Conceptualization; Methodology; Investigation; Writing — Original Draft; Writing — Review & Editing
Romero Méndez, B. S.	Software; Validation; Formal Analysis; Data Curation; Visualization
Zambrano Moreira, A. A.	Resources; Project Administration; Supervision; Funding Acquisition

Cuadro 13: Asignación de roles CRediT.

Declaración sobre la autoría. El historial del repositorio registra tres identidades Git correspondientes a dos personas. La distribución de roles de la tabla 13 refleja el reparto acordado por el equipo, no una medición del historial. Se declara esta discrepancia en lugar de omitirla: quien evalúe el trabajo debe poder contrastar los roles declarados con `git log` y juzgar por sí mismo.

2. Conflicto de intereses

Los autores declaran no tener ningún conflicto de intereses. El trabajo no recibió financiación de ninguna entidad y no existe relación comercial con ninguno de los productos o proveedores citados.

3. Financiación

Este trabajo no recibió financiación externa. Los costes de cómputo se limitaron a los equipos personales de los autores.

4. Disponibilidad de datos y código

El código fuente está disponible en el repositorio citado en la portada, bajo licencia MIT. Los datos de las mediciones (salidas crudas de k6, de la auditoría OWASP, de la auditoría de SQL dinámico, el informe de Lighthouse y el informe de JaCoCo) se publican en `docs/mediciones/`, bajo licencia CC BY 4.0, y su procedencia se documenta en `DATA-PROVENANCE.md`. El conjunto de datos de la aplicación es sintético: no contiene datos de personas reales. No se ha depositado aún en Zenodo y, en consecuencia, **no existe DOI**; se declara esta ausencia en lugar de citar un identificador que no resolvería.

5. Consentimiento informado y ética

No se recogieron datos de participantes humanos. El instrumento SUS se construyó y validó, pero **no se administró a ninguna persona**, de modo que no hubo consentimiento que recabar ni datos personales que tratar. El tratamiento previsto para una futura aplicación del instrumento se describe en `ETHICS.md`. Este trabajo no requirió aprobación de un comité de ética por no involucrar sujetos humanos ni datos personales.

6. Uso de inteligencia artificial

Se emplearon asistentes de programación basados en modelos de lenguaje como apoyo en la redacción de código, de documentación y de este informe. Todo el contenido fue revisado por los autores, que asumen la responsabilidad íntegra sobre él. Ninguna cifra de este informe procede de un modelo de lenguaje: todas se recalculan de los datos crudos versionados, y los procedimientos de recálculo se publican junto a ellos. Las referencias bibliográficas se validaron contra la API de Crossref, DOI por DOI.

7. Limitaciones declaradas

Se declaran, sin atenuantes, las siguientes limitaciones de esta entrega: (i) la usabilidad no se midió, al ser cero el número de participantes del SUS; (ii) no se ejecutó un análisis dinámico de seguridad con OWASP ZAP; (iii) la cobertura de rama es del 16,31 %, muy por debajo de lo deseable; (iv) no se realizó inferencia estadística por insuficiencia de repeticiones; (v) el sistema no está desplegado en un entorno accesible públicamente, y todas las mediciones se tomaron contra `localhost`; (vi) el cribado de trabajos relacionados se limitó a Crossref.

8. Reproducibilidad

La reproducción completa se ejecuta con `make all`. Las cinco imágenes de contenedor están ancladas por digest SHA-256 y el script `scripts/validate-digests.py` verifica el anclaje. El digest de la entrega, que resume el manifiesto de todos los ficheros versionados, figura en la portada y lo genera `scripts/entrega-digest.py`, comprobable con la opción `--check`.

9. Licencia

El código se distribuye bajo licencia MIT; la documentación y los datos de medición, bajo Creative Commons Attribution 4.0 International (CC BY 4.0). Los términos completos figuran en `LICENSE` y en `DATA-PROVENANCE.md`.

A. Anexos

A.1. Anexo A — Bitácora de observaciones

La respuesta a cada observación recibida en las entregas anteriores, con el commit que la resuelve, está en `docs/observaciones/OBSERVACIONES.md`. La bitácora recoge 22 observaciones y 7 hallazgos propios que nadie señaló al equipo.

A.2. Anexo B — Índice de artefactos verificables

Todo lo que este informe afirma puede recomprobarse con los artefactos de la tabla 14, cada uno con el comando que lo regenera.

Artefacto	Cómo comprobarlo
Cobertura de pruebas	<code>docs/mediciones/cobertura/jacoco.csv</code> ; recálculo en §3.1 de <code>COBERTURA.md</code>
Auditoría OWASP	<code>bash scripts/owasp-audit.sh</code> ; crudo en <code>owasp-audit-raw.txt</code>
Auditoría de SQL dinámico	<code>bash scripts/audit-sql-dynamic.sh</code> ; incluye auto-test del instrumento
Carga con k6	<code>scripts/k6-load-test.js</code> ; crudos en <code>docs-/mediciones/perf/</code>
Lighthouse	<code>docs/mediciones/frontend/*.report.json</code>
Trazabilidad	<code>bash scripts/validate-traceability.sh</code>
Índice de ADR	<code>bash scripts/validate-adr.sh</code>
Digests de imagen	<code>python scripts/validate-digests.py</code>
Digest de la entrega	<code>python scripts/entrega-digest.py --check</code>
Diccionario de datos	<code>python scripts/generar-diccionario-datos.py --check</code>
Puntuación SUS	<code>python scripts/sus-score.py</code> (devuelve «sin participantes»)

Cuadro 14: Artefactos verificables y comando que los regenera o comprueba.

A.3. Anexo C — Registros de decisiones de arquitectura

Los nueve ADR están en `docs/adr/`, en formato MADR, cada uno con sus consecuencias negativas explícitas. El script `validate-adr.sh` comprueba que el índice y los ficheros no diverjan.

A.4. Anexo D — Listas de comprobación

Las listas de comprobación de Ralph, FAIR, PRISMA e INCOSE, cumplimentadas ítem por ítem y con la casilla negativa marcada donde corresponde, están en `docs/checklists/`.

Referencias

- [1] ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, 2011.
- [2] ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering, 2018.

- [3] Whisnumurti Adhiwibowo and Ghozali Mahmud. Sistem perpustakaan menggunakan QR code berbasis web dengan framework CodeIgniter. *Information Science and Library*, 2021.
- [4] Scott W. Ambler and Pramod J. Sadalage. *Refactoring Databases: Evolutionary Database Design*. Addison-Wesley, 2006.
- [5] American Library Association. Privacy: An interpretation of the library bill of rights, 2019.
- [6] Association for Computing Machinery. ACM code of ethics and professional conduct, 2018.
- [7] Dhaval Bagal and Pallavi Saindane. Librany — a face recognition and QR code technology based smart library system. In *2019 International Conference on Communication and Electronics Systems (ICCES)*. IEEE, 2019.
- [8] Sebastian Baltes and Paul Ralph. Sampling in software engineering research: A critical review and guidelines. *Empirical Software Engineering*, 27(94), 2022.
- [9] Steven Bingler, Mike West, and John Wilander. Cookies: HTTP state management mechanism. Technical report, Internet Engineering Task Force, 2024. Internet-Draft draft-ietf-httpbis-rfc6265bis.
- [10] Carl Boettiger. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71–79, 2015.
- [11] John Brooke. SUS: A quick and dirty usability scale. In Patrick W. Jordan, Bruce Thomas, Bernard A. Weerdmeester, and Ian L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.
- [12] Simon Brown. *Software Architecture for Developers: Visualise, Document and Explore your Software Architecture*. Leanpub, 2018. Origen del modelo C4.
- [13] Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.
- [14] Kevin P. Fajardo. Smart library system for book cataloging and borrowing using QR codes and OCR. In *2026 International Conference on Machine Learning and Autonomous Systems (ICMLAS)*. IEEE, 2026.
- [15] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. *ACM SIGPLAN Notices*, 42(10):57–76, 2007.
- [16] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [17] Laura Inozemtseva and Reid Holmes. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pages 435–445, 2014.
- [18] Michael B. Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). Technical Report RFC 7519, Internet Engineering Task Force, 2015.
- [19] Santosh Abaji Kharat, Bhausaheb M. Panage, and Shubhada Nagarkar. Use of QR code and Layar app for academic library services. *Library Hi Tech News*, 2017.
- [20] Martin Kleppmann. *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [21] Gerard Meszaros. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley, 2007.

- [22] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of ASPLOS XIV*, pages 265–276, 2009.
- [23] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2 edition, 2021.
- [24] Jakob Nielsen. *Usability Engineering*. Academic Press, 1993.
- [25] Mark Nottingham and Erik Wilde. Problem details for HTTP APIs. Technical Report RFC 7807, Internet Engineering Task Force, 2016.
- [26] Mark Nottingham, Erik Wilde, and Sanjay Dalal. Problem details for HTTP APIs. Technical Report RFC 9457, Internet Engineering Task Force, 2023. Obsoleta la RFC 7807.
- [27] Michael Nygard. Documenting architecture decisions. Cognitect blog, 2011.
- [28] OWASP Foundation. OWASP top 10 — 2021, 2021.
- [29] OWASP Foundation. OWASP API security top 10 — 2023, 2023.
- [30] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- [31] Ken Peffers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77, 2007.
- [32] Bayu Prasetyo, Irsan Jaelani, Dadi Iskandar, and Akbar Saepul Malik. Web-based STT Wastukencana library QR code presence information system. *RISTEC: Research in Information Systems and Technology*, 2024.
- [33] Niels Provos and David Mazières. A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*, pages 81–91, 1999.
- [34] I Gede Sujana Eka Putra, Tirta Mahayana, and Agung Wicaksono. Library management information system using RFID and QR code. *International Journal of Software and Hardware Research in Engineering*, 2021.
- [35] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009.
- [36] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013.
- [37] Rukhsana Shawl. QR code applications in library service provisions. *International Journal of Science and Research (IJSR)*, 2025.
- [38] Heru Supriyono, Muhammad Ramadhan Fitriyan, and Muamaroh. Developing a QR code-based library management system with case study of private school in surakarta city indonesia. In *2018 Third International Conference on Informatics and Computing (ICIC)*. IEEE, 2018.
- [39] Mike West, Adam Barth, and Daniel Veditz. Content security policy level 3. W3C Working Draft, 2024.

- [40] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.
- [41] Hong Zhu, Patrick A. V. Hall, and John H. R. May. Software unit test coverage and adequacy. *ACM Computing Surveys*, 29(4):366–427, 1997.